

TP GSM

Flotte Dylan
Computer Science
HES-SO Master

11 mars 2026

Table des matières

1	Code Python	2
1.1	config.py	2
1.2	crypto.py	2
1.3	session.py	3
1.4	server.py	3
1.5	client.py	6
2	Capture du fonctionnement	9

Chapitre 1

Code Python

1.1 config.py

```
SERVER_HOST = "127.0.0.1"
SERVER_PORT = 5000

# Clé secrète partagée (Ki)
KI = b"SuperSecretKi123"

SESSION_DURATION = 20 * 60 # 20 minutes en secondes
BUFFER_SIZE = 4096
```

1.2 crypto.py

```
import os
import hashlib

# --- RAND ---
def generate_rand():
    return os.urandom(16) # 128 bits

# --- A3 (Simulation SRES) ---
def compute_sres(rand, ki):
    data = ki + rand
    hash_value = hashlib.sha256(data).digest()
    return hash_value[:8] # 64 bits

# --- A8 (Simulation Kc) ---
```

```
def generate_kc(rand, ki):
    data = rand + ki
    hash_value = hashlib.sha256(data).digest()
    return hash_value[:16] # 128-bit session key

# --- XOR Encryption/Decryption ---
def xor_cipher(data, key):
    return bytes([data[i] ^ key[i % len(key)] for i in range(len(data))])
```

1.3 session.py

```
import time
from config import SESSION_DURATION

class Session:
    def __init__(self, kc):
        self.kc = kc
        self.created_at = time.time()

    def is_valid(self):
        return (time.time() - self.created_at) < SESSION_DURATION
```

1.4 server.py

```
import socket
import os
from crypto import generate_rand, compute_sres, generate_kc, xor_cipher
from session import Session
from config import *

def handle_client(conn):
    try:
        print("[SERVER] Client connecté")
        print("[SERVER] Début de l'authentification")

        # 1. Génération RAND
        rand = generate_rand()
        print(f"[SERVER] RAND généré: {rand.hex()[:32]} ({len(rand)} octets)")
        conn.sendall(rand)
```

```
# 2. Réception SRES
client_sres = conn.recv(1024)
print(f"[SERVER] SRES reçu du client: {client_sres.hex()} ({len(client_sres)})

# 3. Vérification
expected_sres = compute_sres(rand, KI)
print(f"[SERVER] SRES attendu: {expected_sres.hex()}")

if client_sres != expected_sres:
    conn.sendall(b"AUTH_FAILED")
    print("[SERVER] Authentification échouée")
    return

conn.sendall(b"AUTH_SUCCESS")
print("[SERVER] Authentification réussie")

# 4. Génération session
kc = generate_kc(rand, KI)
print(f"[SERVER] Clé de session Kc générée: {kc.hex()[:32]} ({len(kc)} octets)
session = Session(kc)
print(f"[SERVER] Session créée à {session.created_at}")

# 5. Réception fichier chiffré
print("[SERVER] Réception du fichier chiffré")
encrypted_data = b""
chunk_count = 0
while True:
    chunk = conn.recv(BUFFER_SIZE)
    if not chunk:
        break
    encrypted_data += chunk
    chunk_count += 1

print(f"[SERVER] Fichier chiffré complet reçu: {len(encrypted_data)} octets")

if not session.is_valid():
    print("[SERVER] Session expirée")
    return
```

```
print("[SERVER] Déchiffrement en cours")
decrypted_data = xor_cipher(encrypted_data, session.kc)
print(f"[SERVER] Fichier déchiffré: {len(decrypted_data)} octets")

with open("files/server_received.mp3", "wb") as f:
    f.write(decrypted_data)

print("[SERVER] Fichier reçu et déchiffré")

# 6. Envoi fichier serveur
print("[SERVER] Lecture du fichier à envoyer")
with open("files/bombinsound-rap-rap-beat-beats-music-20-second-491118.mp3", "rb") as f:
    data = f.read()

print(f"[SERVER] Fichier lu: {len(data)} octets")
print("[SERVER] Chiffrement en cours")
encrypted_response = xor_cipher(data, session.kc)
print(f"[SERVER] Fichier chiffré: {len(encrypted_response)} octets")
print("[SERVER] Envoi du fichier")
conn.sendall(encrypted_response)

print("[SERVER] Fichier envoyé")

except Exception as e:
    print(f"[SERVER ERROR] {e}")
finally:
    conn.close()

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((SERVER_HOST, SERVER_PORT))
    server.listen(1)
    print(f"[SERVER] En écoute sur {SERVER_HOST}:{SERVER_PORT}")

while True:
    conn, addr = server.accept()
    handle_client(conn)
```

```
if __name__ == "__main__":  
    start_server()
```

1.5 client.py

```
import socket  
from crypto import compute_sres, generate_kc, xor_cipher  
from config import *  
  
def start_client():  
    print(f"[CLIENT] Connexion au serveur {SERVER_HOST}:{SERVER_PORT}")  
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
    client.connect((SERVER_HOST, SERVER_PORT))  
    print("[CLIENT] Connecté")  
  
    # 1. Réception RAND  
    rand = client.recv(1024)  
    print(f"[CLIENT] RAND reçu: {rand.hex()[:32]} ({len(rand)} octets)")  
  
    # 2. Calcul SRES  
    print("[CLIENT] Calcul du SRES")  
    sres = compute_sres(rand, KI)  
    print(f"[CLIENT] SRES calculé: {sres.hex()} ({len(sres)} octets)")  
    client.sendall(sres)  
    print("[CLIENT] SRES envoyé au serveur")  
  
    # 3. Vérification résultat  
    response = client.recv(1024)  
    print(f"[CLIENT] Réponse d'authentification: {response.decode('utf-8', errors='ignore')}")  
  
    if response != b"AUTH_SUCCESS":  
        print("[CLIENT] Authentification échouée")  
        return  
  
    print("[CLIENT] Authentification réussie")  
  
    # 4. Génération Kc  
    print("[CLIENT] Génération de la clé de session")
```

```
kc = generate_kc(rand, KI)
print(f"[CLIENT] Clé de session Kc: {kc.hex()[:32]} ({len(kc)} octets)")

# 5. Envoi fichier
print("[CLIENT] Lecture du fichier à envoyer")
with open("files/bombinsound-rap-rap-beat-beats-music-20-second-491118.mp3", "rb") as f:
    data = f.read()

print(f"[CLIENT] Fichier lu: {len(data)} octets")
print("[CLIENT] Chiffrement en cours")
encrypted_data = xor_cipher(data, kc)
print(f"[CLIENT] Fichier chiffré: {len(encrypted_data)} octets")
print("[CLIENT] Envoi du fichier")
client.sendall(encrypted_data)
client.shutdown(socket.SHUT_WR)

print("[CLIENT] Fichier envoyé")

# 6. Réception fichier serveur
print("[CLIENT] Réception du fichier du serveur")
encrypted_response = b""
chunk_count = 0
while True:
    chunk = client.recv(BUFFER_SIZE)
    if not chunk:
        break
    encrypted_response += chunk
    chunk_count += 1

print(f"[CLIENT] Fichier chiffré complet reçu: {len(encrypted_response)} octets")
print("[CLIENT] Déchiffrement en cours")
decrypted_response = xor_cipher(encrypted_response, kc)
print(f"[CLIENT] Fichier déchiffré: {len(decrypted_response)} octets")

with open("files/client_received.mp3", "wb") as f:
    f.write(decrypted_response)

print("[CLIENT] Fichier reçu et déchiffré")
```



```
    client.close()
    print("[CLIENT] Connexion fermée")

if __name__ == "__main__":
    start_client()
```

Chapitre 2

Capture du fonctionnement

```
[SERVER] En écoute sur 127.0.0.1:5000
[SERVER] Client connecté
[SERVER] Début de l'authentification
[SERVER] RAND généré: d2ff3d1935346823425a28cc34f88f36 (16 octets)
[SERVER] SRES reçu du client: 8c44b04316549472 (8 octets)
[SERVER] SRES attendu: 8c44b04316549472
[SERVER] Authentification réussie
[SERVER] Clé de session Kc générée: d1c220e1b4a2ee1dab4d5b062207693d (16
[SERVER] Session créée à 1773069059.1124127
[SERVER] Réception du fichier chiffré
[SERVER] Fichier chiffré complet reçu: 627774 octets
[SERVER] Déchiffrement en cours
[SERVER] Fichier déchiffré: 627774 octets
[SERVER] Fichier reçu et déchiffré
[SERVER] Lecture du fichier à envoyer
[SERVER] Fichier lu: 627774 octets
[SERVER] Chiffrement en cours
[SERVER] Fichier chiffré: 627774 octets
[SERVER] Envoi du fichier
[SERVER] Fichier envoyé
```

FIGURE 2.1 – Capture fonctionnement côté serveur

```
[CLIENT] Connexion au serveur 127.0.0.1:5000
[CLIENT] Connecté
[CLIENT] RAND reçu: d2ff3d1935346823425a28cc34f88f36 (16 octets)
[CLIENT] Calcul du SRES
[CLIENT] SRES calculé: 8c44b04316549472 (8 octets)
[CLIENT] SRES envoyé au serveur
[CLIENT] Réponse d'authentification: AUTH_SUCCESS
[CLIENT] Authentification réussie
[CLIENT] Génération de la clé de session
[CLIENT] Clé de session Kc: d1c220e1b4a2ee1dab4d5b062207693d (16 octets)
[CLIENT] Lecture du fichier à envoyer
[CLIENT] Fichier lu: 627774 octets
[CLIENT] Chiffrement en cours
[CLIENT] Fichier chiffré: 627774 octets
[CLIENT] Envoi du fichier
[CLIENT] Fichier envoyé
[CLIENT] Réception du fichier du serveur
[CLIENT] Fichier chiffré complet reçu: 627774 octets
[CLIENT] Déchiffrement en cours
[CLIENT] Fichier déchiffré: 627774 octets
[CLIENT] Fichier reçu et déchiffré
[CLIENT] Connexion fermée
```

FIGURE 2.2 – Capture fonctionnement côté client